MOBILE APP DEVELOPMENT 700

RICHFIELD
richfield.ac.za

Name and Surname: TINASHE DYLAN TAVEKWA

Student ITS No: 402412247

Qualification: Bsc IT Year of Study: 3 Semester: 2

Assignment due date: 30/10/2026 Date submitted: 30/10/2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

- Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
- Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

- Table of Contents
- Introduction
- Main Body (with relevant subheadings)
- Conclusion
- References

- Submit the assignment in PDF format on Moodle.
- Use the specified cover page provided.
- Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature:  Date: 30/10/2026

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

Table of Contents

..... 1

1.Introduction:..... 3

2. System Design: 3

2.1 Application Structure: 3

2.2 Screen Flow Diagram:..... 4

2.3 Database Design:..... 5

3. Application Screens and Functionality: 5

3.1 Welcome Screen:..... 5

3.2 Home Dashboard: 8

3.3 Pantry Management: 9

3.4 Add Ingredient: 11

3.5 Edit and Delete Ingredient: 13

3.6 Suggested Recipes: 16

3.8 Recipe Details and Cook & Deduct: 18

3.10 Settings:..... 22

4. Key Code Snippets: 23

4.1 SQLite CRUD Operations: 23

4.2 Strict Recipe Matching: 24

4.3 Suggested Recipes: 26

4.4 Cook & Deduct: 27

4.5 Pantry Health: 28

5. Challenges and Solutions 30

5.1 Gradle and Java Version Compatibility: 30

5.2 Duplicate Android Resources; 30

5.3 Strict Recipe Matching: 30

6. Conclusion and Reflection: 31

References:..... 31

1.Introduction:

SmartPantryManager is an Android application developed in Java using Android Studio. The purpose of the application is to help users manage ingredients that they already have at home and reduce unnecessary food waste.A common problem is that people buy food and forget what they already have. Some ingredients then expire before they are used. SmartPantryManager allows users to record ingredients in their pantry together with quantities, units and expiry dates.The application then checks the pantry and suggests recipes that the user can make using only the ingredients currently available. A recipe is only suggested when every required ingredient is available in sufficient quantity. This is the strict recipe-matching rule used by the application.application also includes additional features such as Pantry Health, Pantry Rescue Mode, Cook & Deduct, expiry tracking, meals cooked, ingredients rescued and an All Recipes library.SQLite was used as the main database because the application needs local persistent storage. This means pantry information remains available when the application is closed and reopened.

2. System Design:

2.1 Application Structure:

SmartPantryManager was designed using multiple Android Activities. Each Activity is responsible for a particular part of the application.

The main Activities are:

WelcomeActivity – displays the opening screen.

MainActivity – displays the home dashboard.

PantryActivity – displays ingredients stored in the pantry.

AddIngredientActivity – allows a new ingredient to be added.

EditIngredientActivity – allows an ingredient to be updated.

RecipeActivity – displays recipes that strictly match the current pantry.

AllRecipesActivity – displays the complete recipe library.

RecipeDetailActivity – displays recipe ingredients, method and Cook & Deduct.

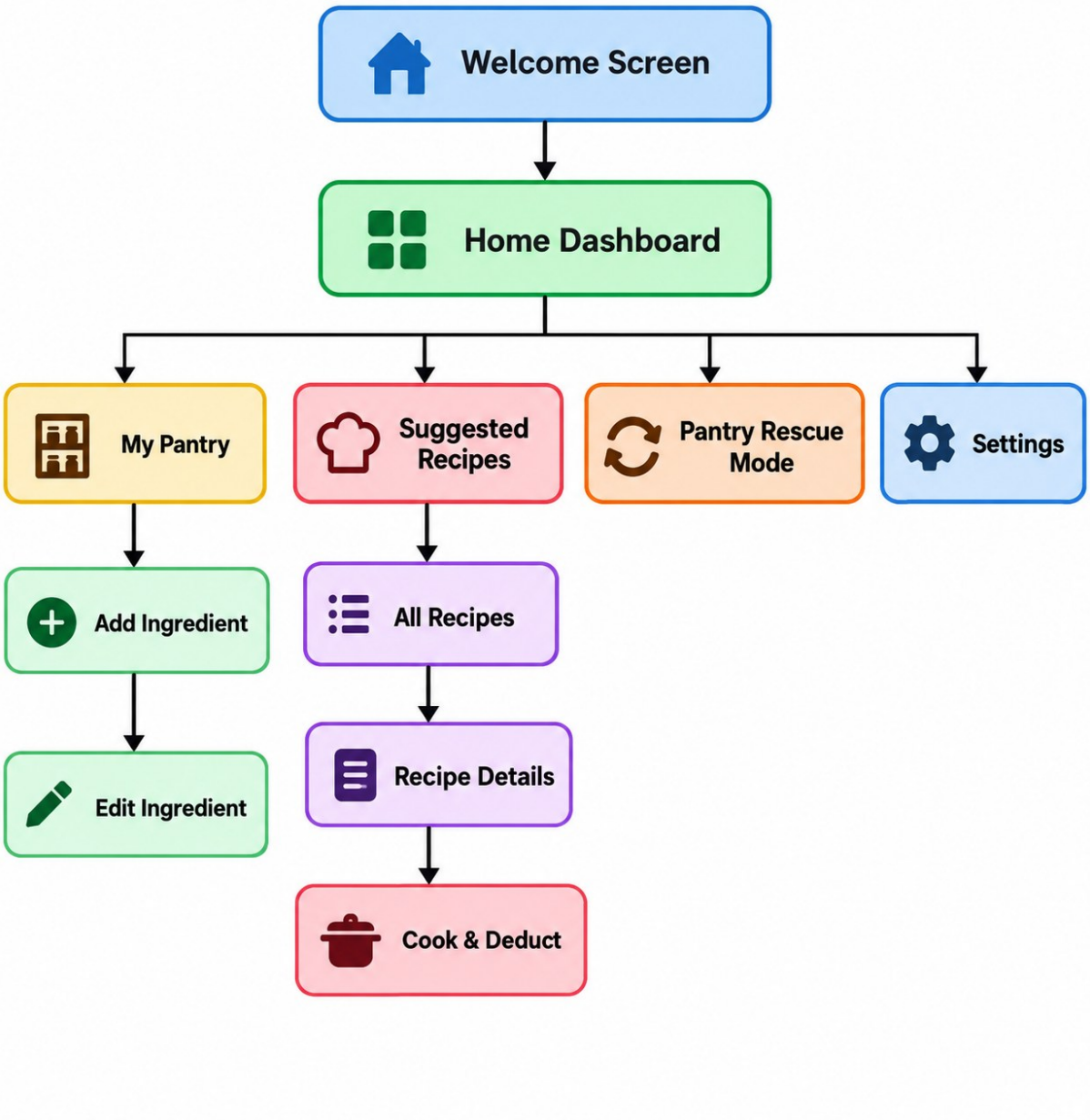
RescueActivity – prioritises recipes using ingredients close to expiry.

SettingsActivity – controls expiry-alert settings.

RecyclerView and custom Adapters are used to display dynamic lists of pantry ingredients and recipes.

SQLite is used for the pantry and recipe data, while SharedPreferences are used for smaller application settings and counters such as expiry alerts, meals cooked and ingredients rescued.

2.2 Screen Flow Diagram:



The bottom navigation bar also allows the user to move between the main Home, Pantry, Recipes, Rescue and Settings sections.

2.3 Database Design:

The application mainly uses three SQLite tables.

Ingredients Table
Stores the ingredients owned by the user.

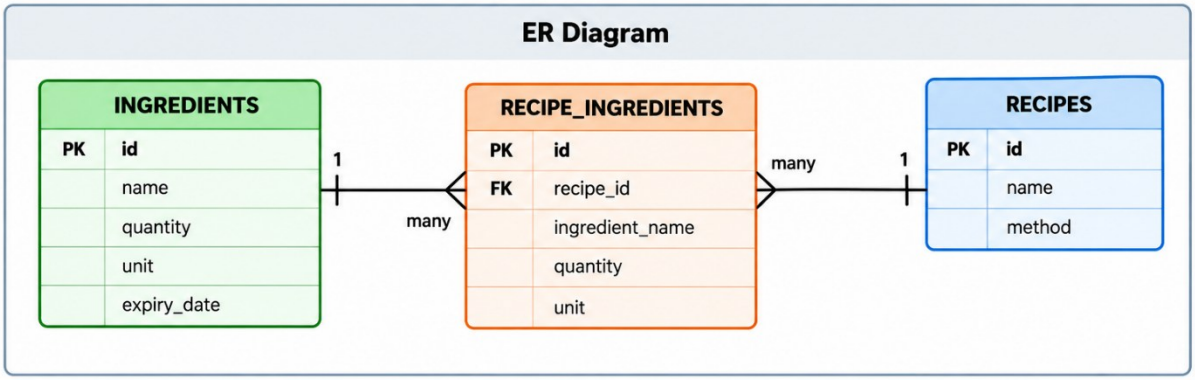
Column	Data Type
id (PK)	INTEGER
name	TEXT
quantity	REAL
unit	TEXT
expiry_date	TEXT

Recipes Table
Stores the main information for each recipe.

Column	Data Type
id (PK)	INTEGER
name	TEXT
method	TEXT

Recipe Ingredients Table
Stores the ingredients required for each recipe.

Column	Data Type
id (PK)	INTEGER
recipe_id (FK)	INTEGER
ingredient_name	TEXT
quantity	REAL
unit	TEXT



The recipes and recipe_ingredients tables are linked by recipe_id.

The pantry ingredients are stored separately. During strict recipe matching, the application compares the requirements stored in recipe_ingredients with the ingredients currently stored in the user's pantry.

3. Application Screens and Functionality:

3.1 Welcome Screen:

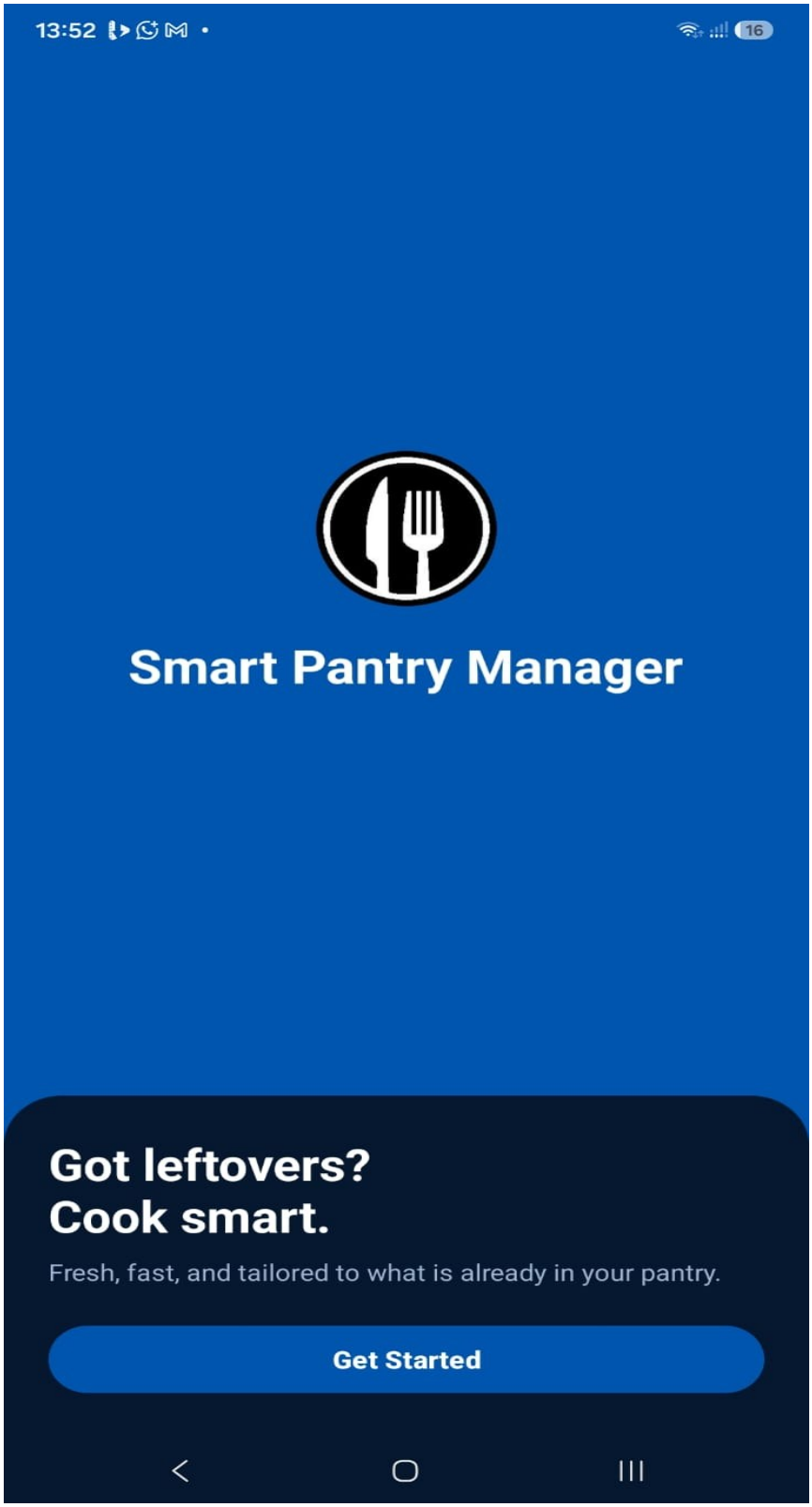
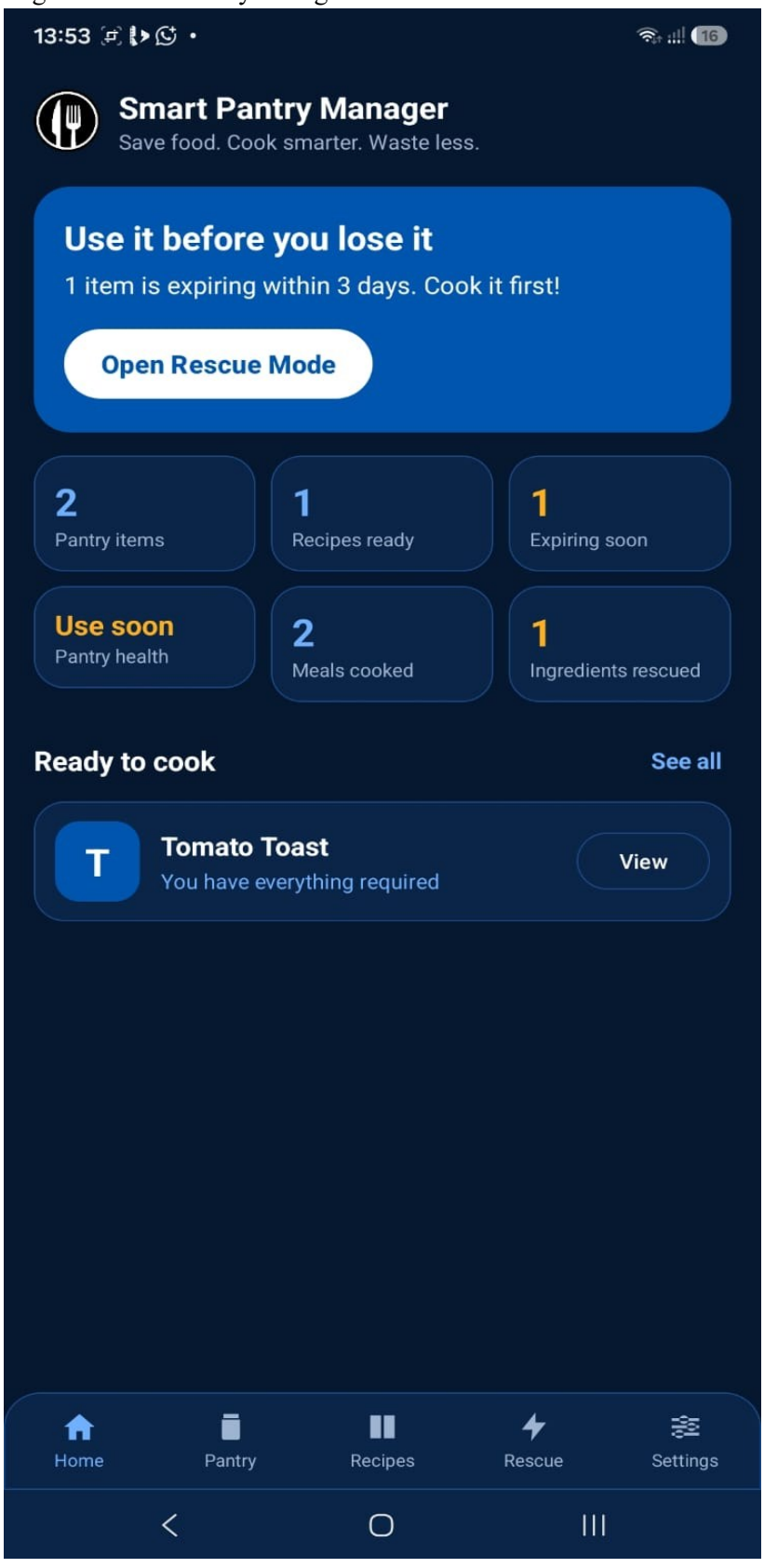

13:52
16
Smart Pantry Manager
Got leftovers?
Cook smart.
Fresh, fast, and tailored to what is already in your pantry.
Get Started

Figure 1: SmartPantryManager Welcome Screen

The Welcome screen is displayed when the application starts. It introduces the application and provides a Get Started button which takes the user to the main dashboard.

3.2 Home Dashboard:

Figure 2: SmartPantryManager Home Dashboard



The Home dashboard gives the user a summary of their pantry.

It displays information such as:

number of pantry items;

number of recipes currently available;

number of ingredients expiring soon;

Pantry Health;

meals cooked; and

ingredients rescued.

Pantry Health gives a simple indication of the condition of the user's pantry.

The possible statuses include:

Empty – no pantry items are stored.

Healthy – no stored food is expired or close to expiry.

Use Soon – one or more ingredients are approaching their expiry date.

High Risk – one or more ingredients have expired.

3.3 Pantry Management:

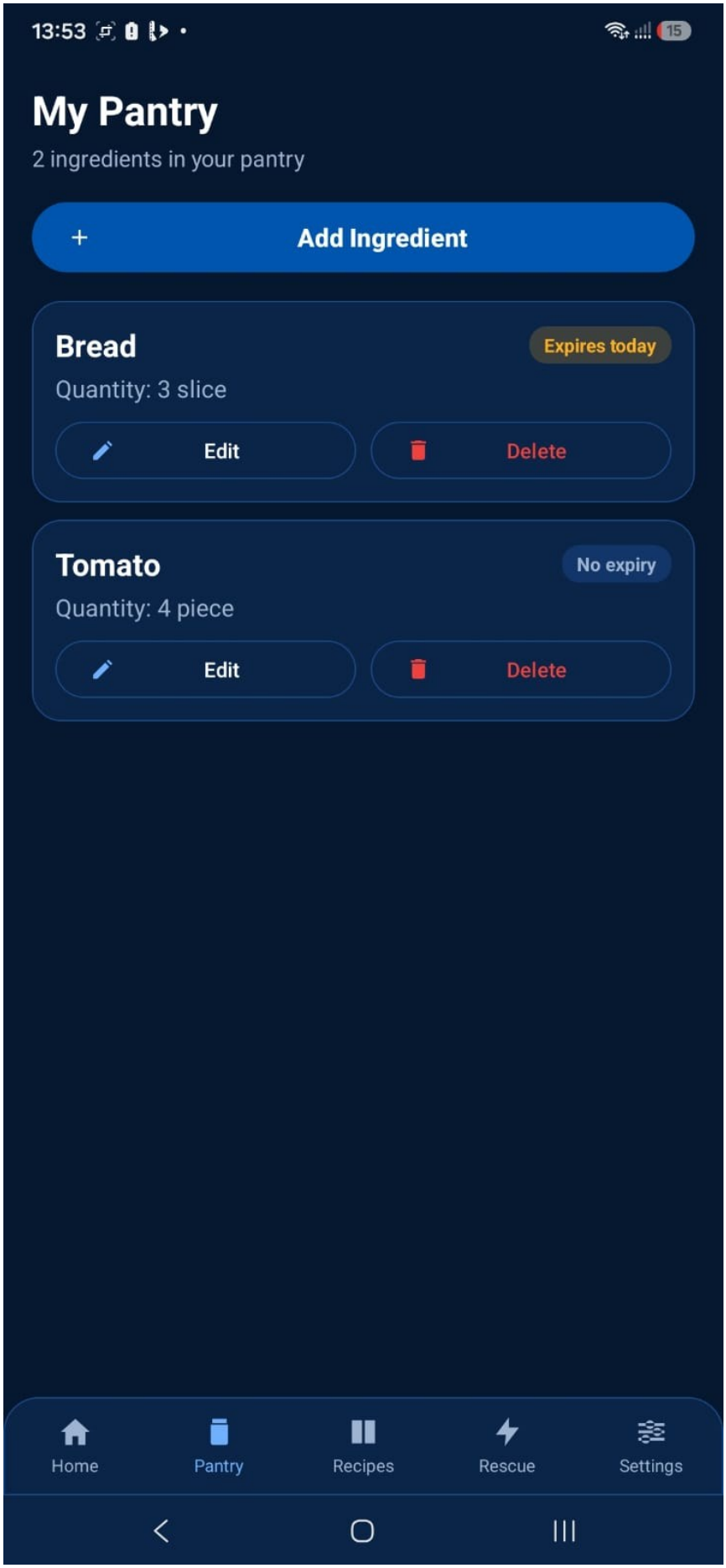

13:53
My Pantry
2 ingredients in your pantry
+ Add Ingredient
Bread
Expires today
Quantity: 3 slice
Edit
Delete
Tomato
No expiry
Quantity: 4 piece
Edit
Delete
Home
Pantry
Recipes
Rescue
Settings

Figure 3: My Pantry Screen

The My Pantry screen displays all ingredients currently stored by the user.

Each pantry item contains information such as:

ingredient name;

quantity;

unit;

expiry date; and

Edit and Delete options.

A RecyclerView is used to display the ingredient records retrieved from SQLite.

3.4 Add Ingredient:

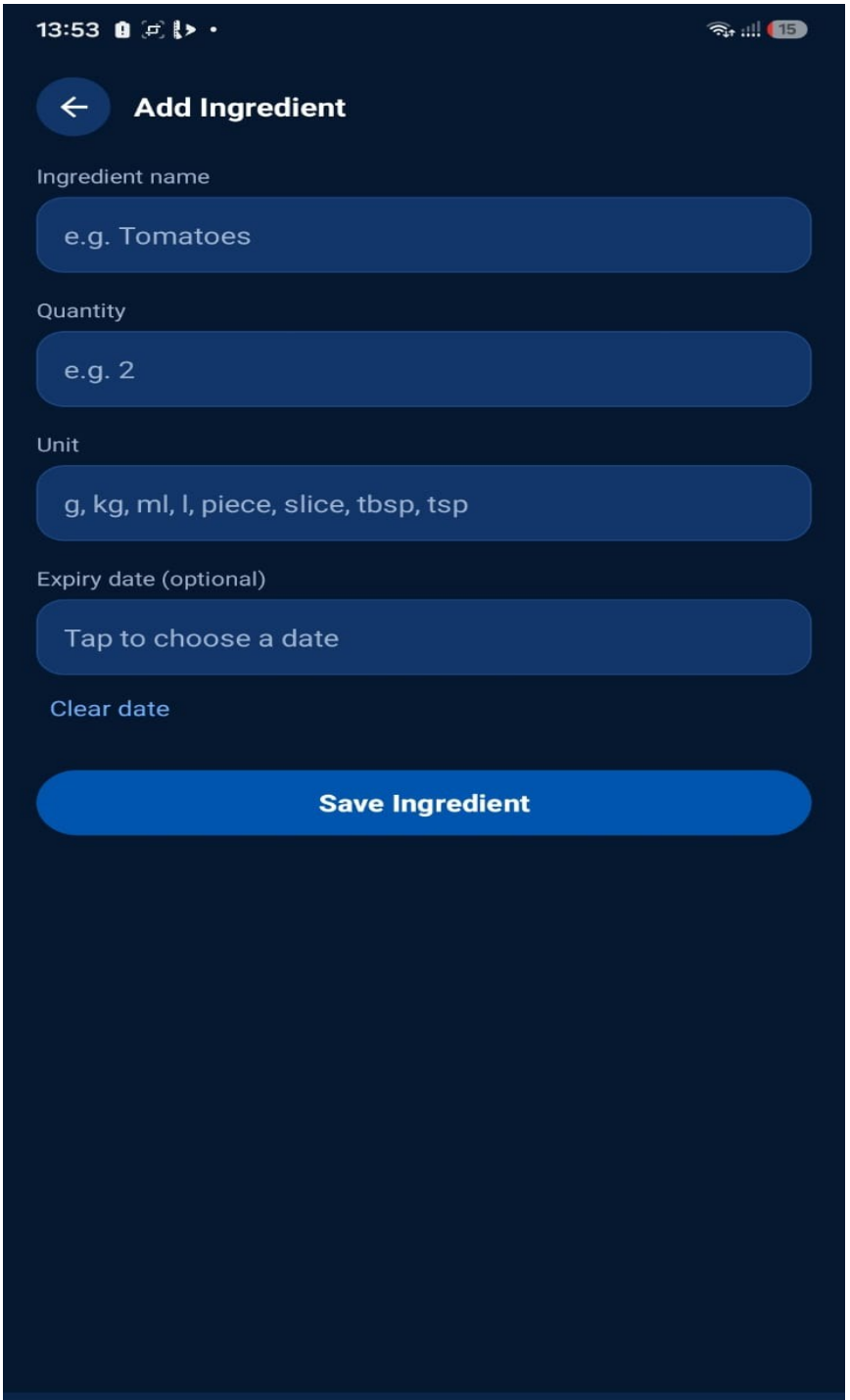

13:53
15
Add Ingredient
Ingredient name
e.g. Tomatoes
Quantity
e.g. 2
Unit
g, kg, ml, l, piece, slice, tbsp, tsp
Expiry date (optional)
Tap to choose a date
Clear date
Save Ingredient

Figure 4: Add Ingredient Screen

The Add Ingredient screen allows the user to create a new pantry record.

The user enters the ingredient name, quantity, unit and expiry date.

Input validation prevents incorrect or incomplete information from being saved.

For example, the application does not allow the user to save an ingredient without providing a name and valid quantity.

[INSERT SCREENSHOT – VALIDATION ERROR]

Figure 5: Input Validation

This demonstrates the validation displayed when required information is missing.

3.5 Edit and Delete Ingredient:

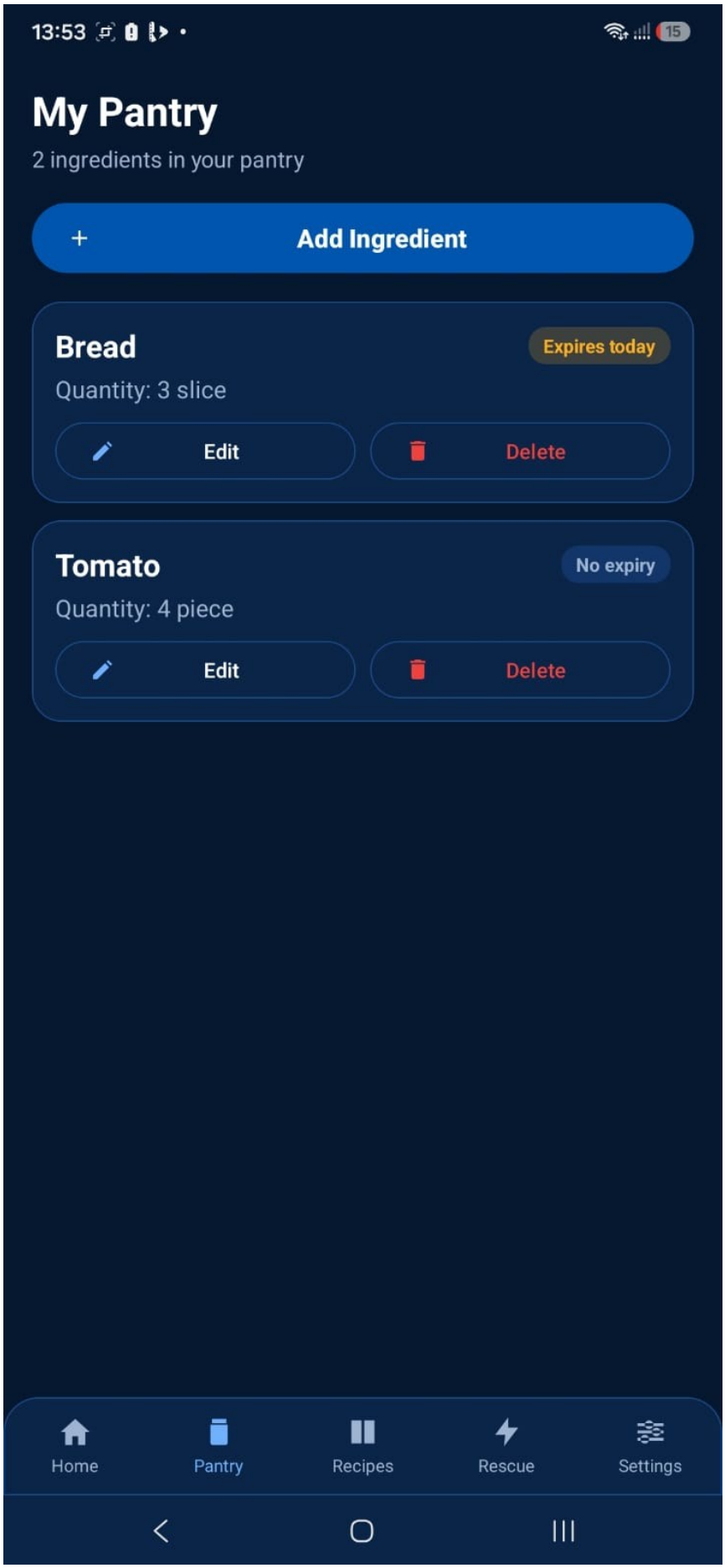

13:53
15
My Pantry
2 ingredients in your pantry
+
Add Ingredient
Bread
Expires today
Quantity: 3 slice
Edit
Delete
Tomato
No expiry
Quantity: 4 piece
Edit
Delete
Home
Pantry
Recipes
Rescue
Settings

Figure 6: Edit Ingredient Screen

The Edit function allows an existing ingredient to be changed.

For example, if the quantity of bread changes from two slices to six slices, the user can edit the pantry record and save the new quantity.

The Delete option removes an ingredient from the SQLite database. A confirmation message is displayed to reduce accidental deletion.

Together, Add, Read, Edit and Delete demonstrate full CRUD functionality.

3.6 Suggested Recipes:

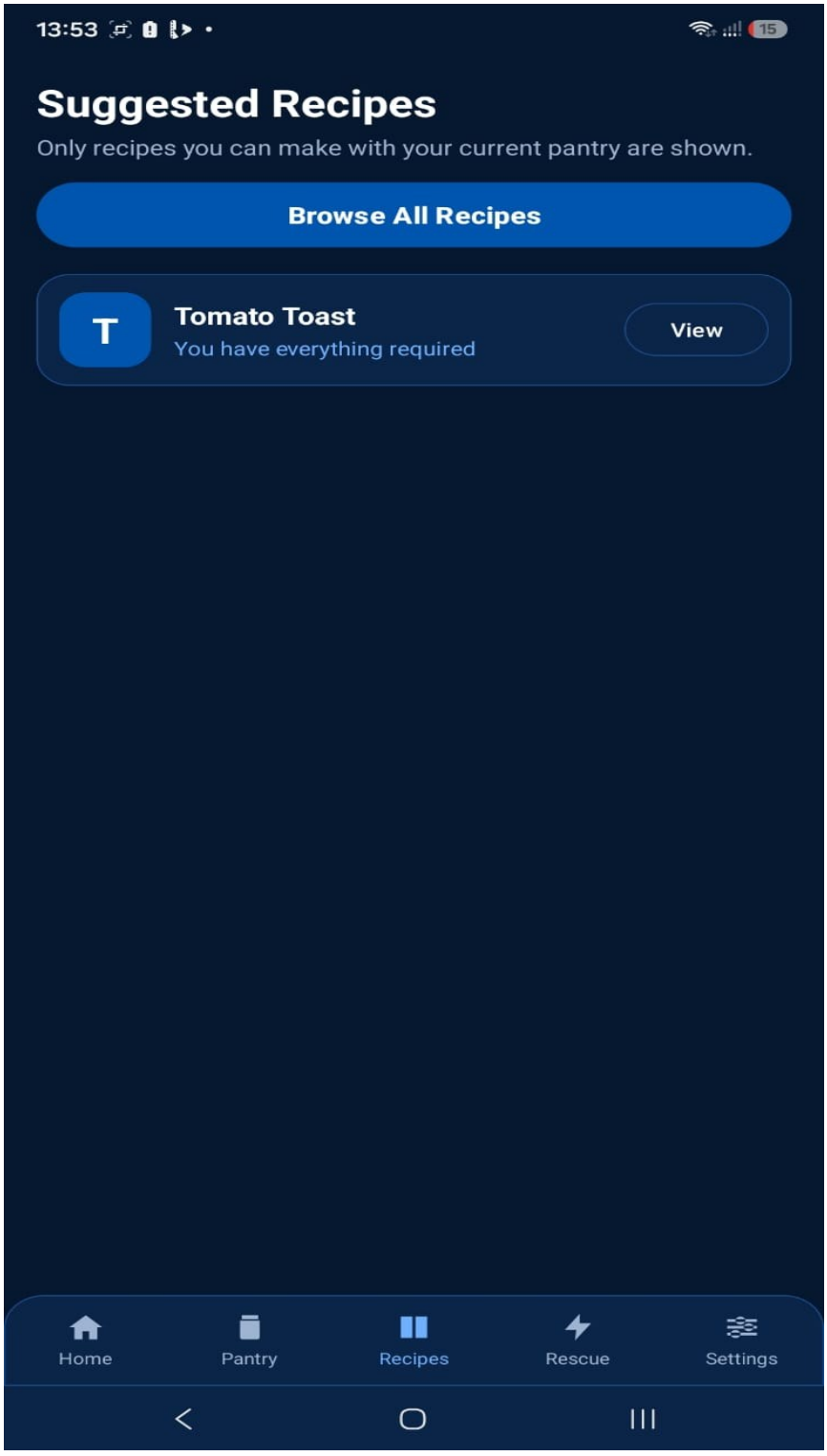


Figure 8: Suggested Recipes Screen

Suggested Recipes is one of the main features of SmartPantryManager.

The screen only displays recipes that pass the strict-matching rule.

For every recipe, the application checks:

whether every required ingredient exists in the pantry;

whether the pantry quantity is sufficient; and

whether compatible units can be compared.

If even one required ingredient is missing or insufficient, the recipe is excluded from Suggested Recipes.

For example, if Tomato Toast requires bread and tomato but the pantry only contains bread, Tomato Toast will not be suggested.

3.7 All Recipes:

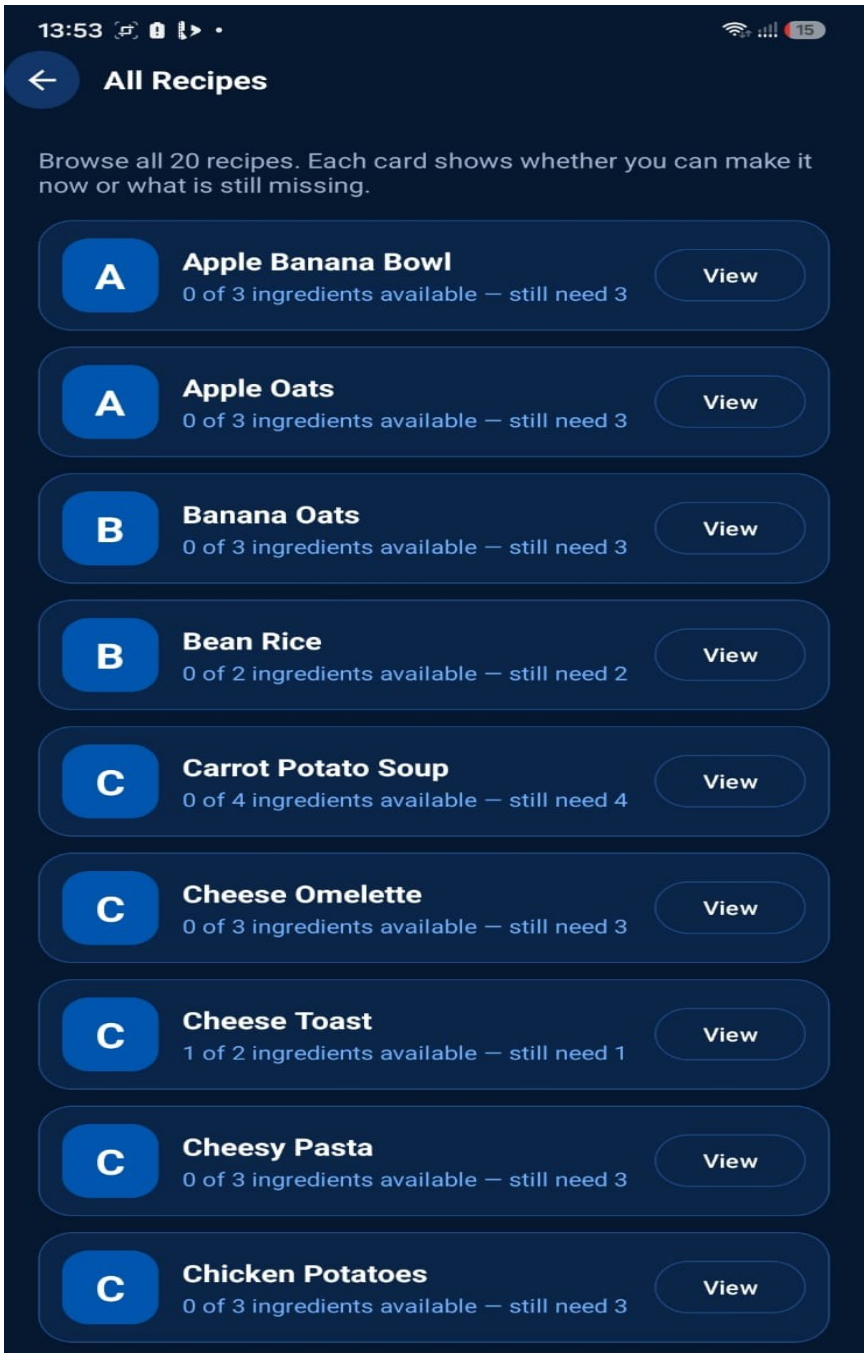


Figure 9: All Recipes Library

The All Recipes screen displays the complete recipe collection.

This is separate from Suggested Recipes.

Suggested Recipes only contains recipes that the user can make immediately, while All Recipes allows the user to browse the entire recipe library.

The application contains at least 20 seeded recipes.

The recipe cards can also indicate whether a recipe is currently ready or whether ingredients are still missing.

3.8 Recipe Details and Cook & Deduct:

Figure 10: Recipe Details

The Recipe Details screen displays:

recipe name;

complete required ingredient list;

quantity and unit for each ingredient;

ingredient availability;

cooking method; and

Cook This Recipe button.

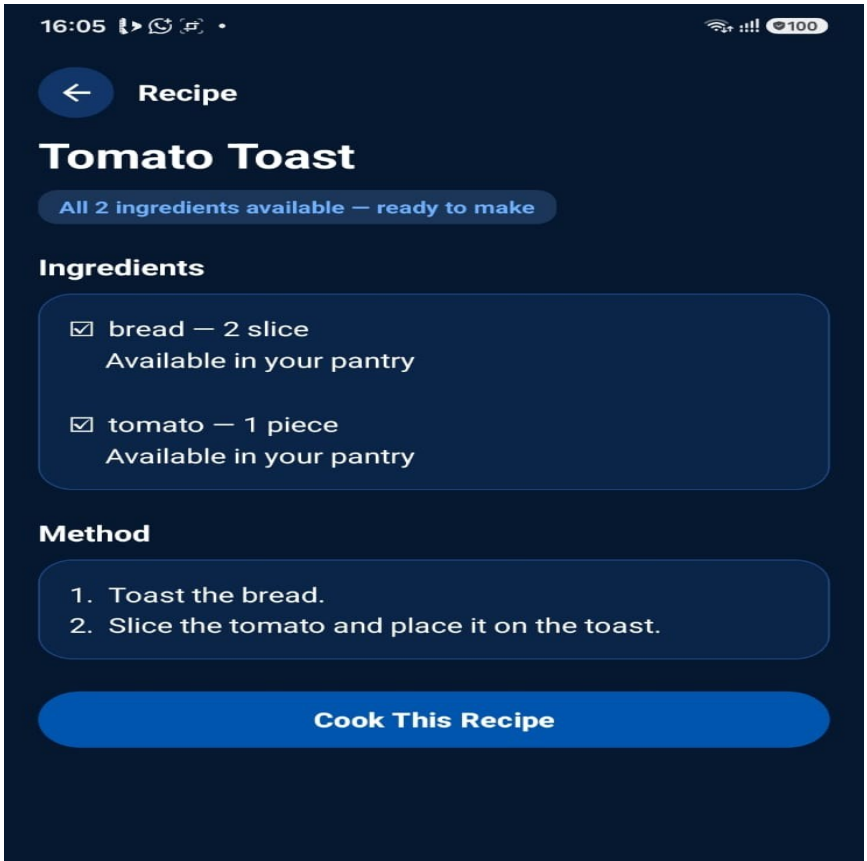


Figure 11: Cook & Deduct Feature

When the user selects Cook This Recipe, the application checks the pantry again before making any changes.

If all required ingredients are still available, the required quantities are automatically deducted from the pantry.

Ingredients that are close to expiry are used first where possible.

After cooking:

the pantry quantities are updated;

meals cooked increases;

rescued ingredients can increase;

Suggested Recipes are recalculated; and

the application can display another recipe that may be made with the remaining ingredients.

This feature helps keep the digital pantry consistent with ingredients that have actually been used.3.9

Pantry Rescue Mode:


13:53
15
Pantry Rescue
Recipes that use ingredients expiring within 3 days come first.
T
Tomato Toast
Rescue score: 1 expiring ingredient
View
Home
Pantry
Recipes
Rescue
Settings

Figure 12: Pantry Rescue Mode

Pantry Rescue Mode is designed to reduce food waste.

It first uses the same strict-matching rule as Suggested Recipes. This means Rescue Mode never recommends a recipe that the user cannot fully make.

The application then checks which matching recipes use ingredients that are approaching their expiry date.

A Rescue Score is calculated based on how many expiring pantry ingredients are required by the recipe.

Recipes with a higher Rescue Score are displayed with higher priority.

3.10 Settings:

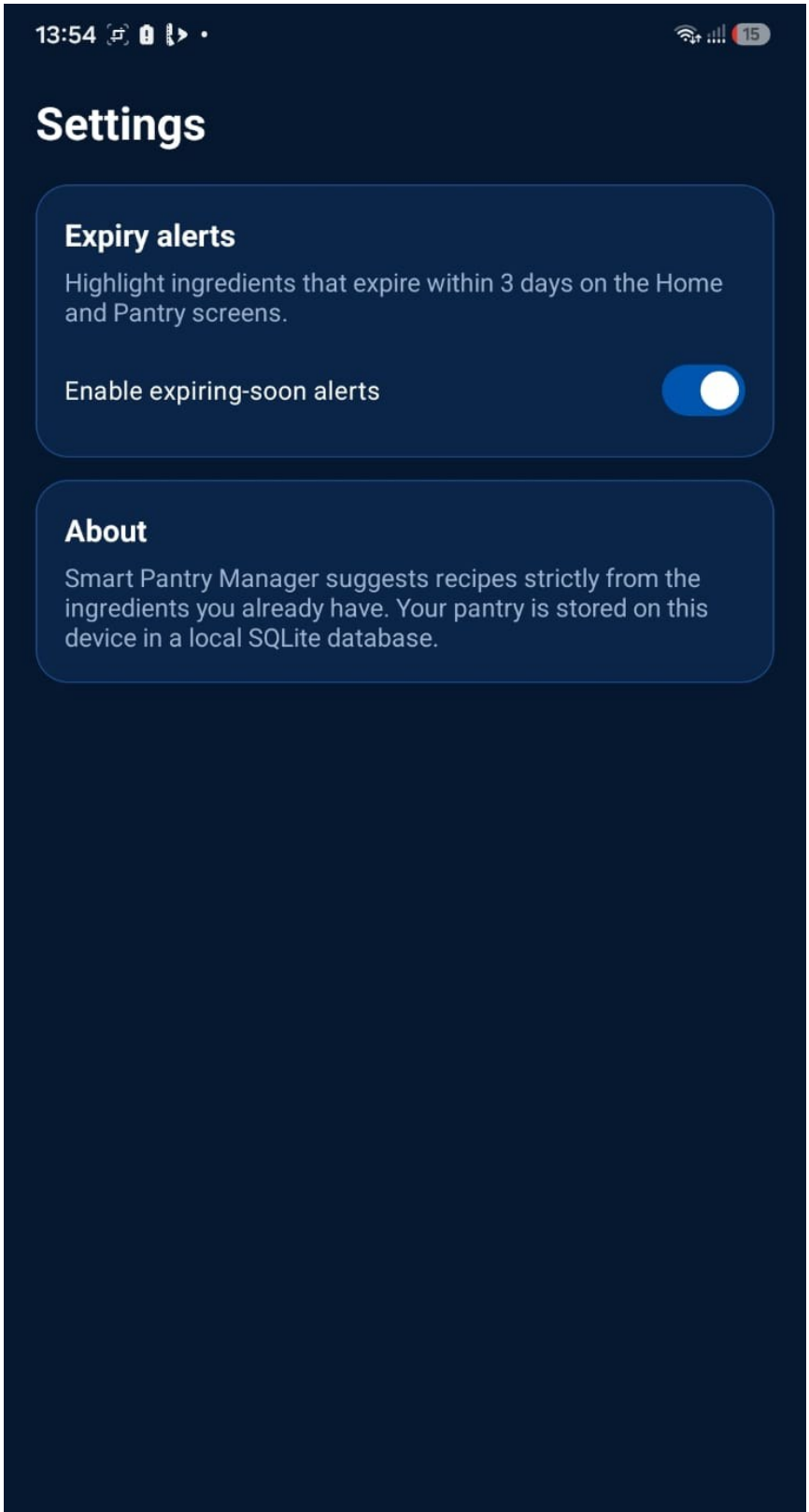


Figure 13: Settings Screen

The Settings screen allows the user to enable or disable expiry alerts.

The preference is stored using SharedPreferences so that it remains saved when the user leaves the screen or reopens the application.

4. Key Code Snippets:

4.1 SQLite CRUD Operations:

The following methods are part of DatabaseHelper.java.

```
public boolean addIngredient(Ingredient ingredient) {

    SQLiteDatabase db = getWritableDatabase();

    return db.insert(
            TABLE_INGREDIENTS,
            null,
            toValues(ingredient)
    ) != -1;
}

public boolean updateIngredient(Ingredient ingredient) {

    SQLiteDatabase db = getWritableDatabase();

    int rows = db.update(
            TABLE_INGREDIENTS,
            toValues(ingredient),
            COL_ID + "=?",
```

```
            new String[]{String.valueOf(ingredient.getId())}
    );

    return rows > 0;
}

public boolean deleteIngredient(int id) {

    SQLiteDatabase db = getWritableDatabase();

    int rows = db.delete(
            TABLE_INGREDIENTS,
            COL_ID + "=?",
            new String[]{String.valueOf(id)}
    );

    return rows > 0;
}
```

Explanation:

These methods are used to Create, Update and Delete pantry records. SQLite provides persistent storage, which means the pantry information remains available after the application is closed.

4.2 Strict Recipe Matching:

The main matching logic is contained inside RecipeMatcher.

```
public static boolean canMake(
        List<RecipeRequirement> requirements,
        List<Ingredient> pantry) {
```

```
    if (requirements == null || requirements.isEmpty()) {
        return false;
    }

    for (RecipeRequirement requirement : requirements) {

        double available =
                availableQuantity(pantry, requirement);

        double required =
                toBaseQuantity(
                        requirement.getQuantity(),
                        requirement.getUnit()
                );

        if (available + 1e-9 < required) {
            return false;
        }
    }

    return true;
}
```

Explanation:

The method checks each requirement for a recipe. If one required ingredient is missing or there is not enough quantity, the method immediately returns false.

The recipe is only accepted when every requirement passes.

The application also normalises ingredient names and compatible units. For example, egg and eggs can match, while kilograms can be converted to grams and litres can be converted to millilitres.

4.3 Suggested Recipes:

```
public ArrayList<Recipe> getSuggestedRecipes() {


   ArrayList<Recipe> suggested =

         new ArrayList<>();


   ArrayList<Ingredient> pantry =

         getAllIngredients();


   for (Recipe recipe : getAllRecipes()) {


      if (RecipeMatcher.canMake(

            getRecipeRequirements(recipe.getId()),

            pantry)) {


         suggested.add(recipe);

      }

   }


   return suggested;

}
```

Explanation:

This method retrieves all recipes and passes each recipe through the strict-matching algorithm.

Only recipes that return true are added to the Suggested Recipes list.

This ensures that partial matches are not displayed on the main suggestions screen.

4.4 Cook & Deduct:

```
private void cookRecipe() {


    DatabaseHelper.CookResult result =
            databaseHelper.consumeRecipe(recipeId);


    if (!result.isSuccess()) {


        Toast.makeText(
                this,
                R.string.cook_failed,
                Toast.LENGTH_SHORT
        ).show();


        refreshRecipeState();
        return;
    }


    AppSettings.recordCookedMeal(
            this,
            result.getRescuedIngredients()
    );
```

```
    Toast.makeText(
            this,
            R.string.cook_success,
            Toast.LENGTH_SHORT
    ).show();

    refreshRecipeState();
}
```

Explanation:

When Cook This Recipe is selected, consumeRecipe() checks whether the recipe can still be made.

If successful, the required quantities are deducted from the pantry. The application then records the meal and any expiring ingredients that were rescued.

The recipe state is refreshed after cooking because the remaining pantry quantities may change which recipes can still be made.

4.5 Pantry Health:

```
private void updatePantryHealth(
        int itemCount,
        int expiring,
        int expired) {

    if (itemCount == 0) {

        txtPantryHealth.setText(
                R.string.health_empty
        );
```

```
    } else if (expired > 0) {

        txtPantryHealth.setText(
                R.string.health_high_risk
        );

    } else if (expiring > 0) {

        txtPantryHealth.setText(
                R.string.health_use_soon
        );

    } else {

        txtPantryHealth.setText(
                R.string.health_healthy
        );
    }
}
```

Explanation:

Pantry Health provides a simple status based on the user's stored food.

Expired ingredients have the highest priority, followed by ingredients that are close to expiry. If no problem is detected, the pantry is shown as Healthy.

5. Challenges and Solutions

5.1 Gradle and Java Version Compatibility:

One challenge during development was a Gradle JVM compatibility problem.

The project used Gradle 8.9 while the computer was initially configured to use Java 25. The selected Java version was too new for that Gradle configuration.

The problem was solved by changing the Gradle JDK to Java 21, which was compatible with the project.

This taught me that the Java version used by Android Studio and the version supported by Gradle need to be compatible.

5.2 Duplicate Android Resources;

Another problem occurred after updating the project files.

Both .png and .webp versions of the launcher icon existed with the same Android resource name. This caused the build to fail because Android detected duplicate resources.

The duplicate .webp launcher files were removed and the new PNG icons were retained.

After cleaning and rebuilding the project, the application built successfully.

This taught me to be careful when merging resource folders because Windows can keep old files even when newer files are copied into the same directory.

5.3 Strict Recipe Matching:

A more important programming challenge was making recipe matching strict but still practical.

A basic text comparison would treat values such as tomato and tomatoes as different ingredients. It could also fail when one quantity is entered in kilograms and another is stored in grams.

I solved this by creating a separate RecipeMatcher class.

The class normalises ingredient names and converts compatible units into common base units before quantities are compared.

This allowed the application to remain strict while still handling simple real-life differences in how ingredients are entered.

6. Conclusion and Reflection:

SmartPantryManager achieved the main aim of the project by allowing users to manage pantry ingredients and receive recipe suggestions based only on food they currently have.

The application supports full CRUD functionality using SQLite and stores pantry information persistently.

The strict recipe-matching system ensures that a recipe is only suggested when all required ingredients are available in sufficient quantities.

Additional features such as Pantry Health, expiry tracking, Pantry Rescue Mode, Rescue Scores, Cook & Deduct, All Recipes, meals cooked and ingredients rescued improved the usefulness of the application.

During this project, I improved my understanding of Java and Android development. I gained practical experience using Activities, Intents, RecyclerViews, custom Adapters, SQLite, SharedPreferences and GitHub version control.

I also learned more about debugging Android projects, particularly Gradle compatibility, resource conflicts and testing application logic.

If I had more time, I would improve the application further by adding optional user accounts, more recipes, better accessibility options and more advanced expiry notifications. However, I would keep the strict recipe-matching rule because it is the main feature that makes the application useful for reducing food waste.

Overall, the project helped me understand how different Android concepts work together to create a complete mobile application.

References:

Android Developers (2026) *Create dynamic lists with RecyclerView*. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview (Accessed: 30 October 2026).

Android Developers (2026) *Save data using SQLite*. Available at: https://developer.android.com/training/data-storage/sqlite (Accessed: 30 October 2026).

Android Developers (2026) *Intents and intent filters*. Available at: https://developer.android.com/guide/components/intents-filters (Accessed: 30 October 2026).

Android Developers (2026) *Save simple data with SharedPreferences*. Available at: https://developer.android.com/training/data-storage/shared-preferences (Accessed: 30 October 2026).

Git (2026) *Git documentation*. Available at: https://git-scm.com/doc (Accessed: 30 October 2026).

Oracle (2026) *Java documentation*. Available at: https://docs.oracle.com/en/java/ (Accessed: 30 October 2026).